PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PROSEDUR IMPERATIF TEROTOMASI DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A CONTROLLED COMPARISON OF AUTOMATED IMPERATIVE PROCEDURES AND GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PROSEDUR IMPERATIF TEROTOMASI DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

A CONTROLLED COMPARISON OF AUTOMATED IMPERATIVE PROCEDURES AND GITOPS FOR ADDRESSING CONFIGURATION DRIFT IN KUBERNETES CLUSTERS

Usulan Penelitian



MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

PROGRAM STUDI ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

HALAMAN PENGESAHAN

PROPOSAL TUGAS AKHIR

PERBANDINGAN TERKONTROL PROSEDUR IMPERATIF TEROTOMASI DAN GITOPS DALAM MENGHADAPI PENYIMPANGAN KONFIGURASI PADA KLASTER KUBERNETES

Telah dipersiapkan dan disusun oleh

MUHAMMAD ARGYA VITYASY
23/522547/PA/22475

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Juni 2026

Susunan Tim Penguji

Guntur Budi Herwanto, S.Kom., M.Cs. Pembimbing	Dr. techn. Kabul Kurniawan, S.Kom., M.Cs. Ketua Penguji

Aina Musdholifah, S.Kom., M.Kom., Ph.D.
Anggota Penguji

DAFTAR ISI

Halaman Judul **ii**

Halaman Pengesahan **iii**

INTISARI **vi**

ABSTRACT **vii**

I PENDAHULUAN **1**

1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 5
1.5 Manfaat Penelitian 5
1.6 Sistematika Penulisan 5

II PENELITIAN TERKAIT **7**

2.1 Penyimpangan Konfigurasi (*Configuration Drift*) 7
2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi 8
2.2 Terminologi Dasar 8
2.2.1 Kubernetes dan Orkestrasi Kontainer 8
2.2.2 Git dan *Version Control* 9
2.2.3 *Continuous Integration* dan *Continuous Deployment* 9
2.2.4 *Infrastructure-as-Code* (IaC) 9
2.2.5 Paradigma Deklaratif vs Imperatif 9
2.3 Manajemen Konfigurasi Infrastruktur 10
2.4 *Continuous Deployment* dan Paradigma GitOps 10
2.4.1 Performa *Deployment* dan *Anti-Pattern* 11
2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif . 11
2.5.1 *Convergence* Menuju *Desired State* 12
2.5.2 *Audit Trail* sebagai *Social Control Mechanism* 12
2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan 12
2.7 Paradigma *Deployment* Alternatif 13
2.8 Evaluasi Empiris GitOps dan ArgoCD 13

2.9 Metode Eksperimental Terkontrol pada Riset Sistem 15
2.10 Klasifikasi Beban Uji (*Workload*) 15
2.11 Kesenjangan Penelitian (*Research Gap*) 16

III METODE DAN RANCANGAN PENELITIAN 18
3.1 Deskripsi Umum dan Status Rancangan 18
3.2 Metodologi dan Kesetaraan Perbandingan 18
3.3 Lingkungan dan Provenans Deployment 21
3.4 Intervensi dan Kepemilikan Pengendali 22
3.5 Variabel dan Definisi Operasional 24
3.5.1 R1: Kesetaraan Konfigurasi 24
3.5.2 R2: Deteksi, Bukti Deviasi, dan Pemulihan 24
3.5.3 R3: Batas Pengukuran dan Pekerjaan Lanjutan 25
3.6 Gerbang Baseline dan Reset 26
3.7 Instrumen dan Protokol Pengukuran 27
3.8 Pengulangan dan Analisis Statistik 28
3.9 Ancaman terhadap Validitas 29
3.9.1 Validitas Konstruk 29
3.9.2 Validitas Internal 29
3.9.3 Validitas Eksternal 29
3.9.4 Validitas Kesimpulan 29

IV JADWAL PENELITIAN 31
4.1 Penjelasan Kegiatan 31

INTISARI

Perbandingan Terkontrol Prosedur Imperatif Terotomasi dan GitOps dalam Menghadapi Penyimpangan Konfigurasi pada Klaster Kubernetes

Oleh

Muhammad Argya Vityasy
23/522547/PA/22475

Penelitian ini merancang studi kasus komparatif terkontrol atas dua mode pengelolaan pada satu stack InvenioRDM produksi-like: mode imperatif terotomasi berbasis `kubectl` dan mode GitOps berbasis ArgoCD. Pada mode imperatif, *auto-sync*/*self-heal* ArgoCD dijeda, pemeriksa dan *runbook* otomatis menjalankan koreksi `kubectl`, lalu ArgoCD dipulihkan; ini bukan *deployment* terpisah dan bukan studi waktu reaksi manusia. Kedua mode berjalan berurutan pada stack yang sama; tidak ada stack kembar `exp-*` dan tidak ada eksekusi paralel A/B, sehingga non-isolasi terhadap beban latar dan dependensi bersama menjadi keterbatasan eksplisit. Beban formal adalah InvenioRDM nyata dengan revisi sumber, render, image digest, dan versi runtime yang dibekukan. Tujuh skenario direncanakan. Skenario A adalah kontrol HPA dan `/spec/replicas` dikecualikan dari R1 statis; F adalah batas visibilitas objek tak terkelola; G adalah kasus deskriptif yang bukan *self-heal* ArgoCD. A, F, dan G dipisahkan dari estimasi efikasi B–E. Metrik mencakup kesetaraan bidang konfigurasi (R1) serta waktu deteksi, ketersediaan bukti deviasi, dan pemulihan konfigurasi (R2); kesehatan dilaporkan terpisah. R3 tidak diukur karena audit API server tidak tersedia dan tidak dilaporkan sebagai nol atau tidak ada perbedaan. Sampel formal adalah $7 \times 2 \times 15 = 210$ run (10 R2 dan 5 R1 per sel) dengan sampling satu detik. Analisis mempertimbangkan uji nonparametrik berpasangan/blok, ukuran efek dan interval kepercayaan, serta penanganan tersensor kanan; studi ini adalah studi kasus tunggal sehingga validitas eksternalnya terbatas. Rancangan masih menunggu tinjauan dan belum ada data formal.

Kata Kunci: penyimpangan konfigurasi, Kubernetes, GitOps, prosedur imperatif terotomasi, studi kasus komparatif terkontrol

ABSTRACT

A Controlled Comparison of Automated Imperative Procedures and GitOps for Addressing Configuration Drift in Kubernetes Clusters

By

Muhammad Argya Vityasy
23/522547/PA/22475

This proposal designs a controlled single-case comparative study of two management modes on one production-like InvenioRDM stack: an automated imperative mode based on kubectl and a GitOps mode based on ArgoCD. In the imperative mode, ArgoCD auto-sync/self-heal is paused, an automated checker and runbook perform kubectl corrections, and ArgoCD is then restored; this is not a separate deployment and not a human-reaction-time study. Both modes run sequentially on the same stack; there is no exp-* twin and no parallel A/B execution, so non-isolation from background load and shared dependencies is an explicit limitation. The formal workload is real InvenioRDM with a frozen source revision, render, image digests, and runtime versions. Seven scenarios are planned. Scenario A is an HPA control and /spec/replicas is excluded from static R1; F is a visibility boundary for unmanaged objects; G is a descriptive case that is not an ArgoCD self-heal case. A, F, and G are separated from efficacy estimates for B–E. Metrics cover selected-field configuration equality (R1) and detection, deviation-evidence availability, and configuration recovery times (R2); health is reported separately. R3 is not measured because Kubernetes API-server audit is unavailable and is not reported as zero or as no difference. The formal sample is 7 x 2 x 15 = 210 runs (10 R2 and 5 R1 per cell) with one-second sampling. Analysis considers paired/blocked nonparametric tests, effect sizes and confidence intervals, and explicit right-censoring; the study is a single case, so external validity is limited. Owner and supervisor review remains pending, and no formal dataset is available.

Keywords: configuration drift, Kubernetes, GitOps, automated imperative procedure, controlled single-case comparative study

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kubernetes telah menjadi platform *container orchestration* yang dominan di industri maupun akademisi, digunakan oleh lebih dari 5,6 juta *developer* secara global (Cloud Native Computing Foundation, 2024). Kemampuan Kubernetes dalam mengelola *deployment*, *scaling*, dan *self-healing* menjadikannya pilihan utama untuk menjalankan aplikasi yang memerlukan *availability* tinggi (Burns et al., 2016). Namun, seiring bertambahnya jumlah layanan yang di-*deploy* pada sebuah klaster, kompleksitas operasional meningkat secara drastis. Aplikasi yang dijalankan pada Kubernetes mencakup berbagai kelas *workload* dengan karakteristik yang berbeda, antara lain: (1) *workload* web tanpa state (*stateless web microservice*), (2) *workload* berbasis data yang menyimpan state (*stateful data service*), serta (3) *workload* komposit multi-service yang terdiri atas banyak komponen yang saling bergantung (Siddiqui et al., 2023; Aghili et al., 2024). Kelas terakhir, seperti platform data repositori skala besar, dapat terdiri dari 16 atau lebih komponen: layanan *web*, *worker* asinkron, *database* relasional, *search engine*, *cache*, *object storage*, *ingress controller*, *certificate manager*, enkripsi *secret*, *tunnel* jaringan, kebijakan keamanan, *monitoring system*, dan *backup*.

Pengelolaan *deployment* aplikasi multi-service menggunakan pendekatan manual, yakni melalui perintah `kubectl apply`, *manual scaling*, atau *patching* langsung pada klaster, sering menghasilkan inkonsistensi operasional. Praktik ini rentan terhadap penyimpangan konfigurasi (*configuration drift*), yaitu situasi di mana *state* aktual klaster menyimpang dari *state* yang didefinisikan secara deklaratif (Zhang et al., 2026). Penyimpangan konfigurasi dapat timbul akibat tiga penyebab utama: (1) perubahan langsung pada klaster yang tidak tercatat dalam sistem *version control*; (2) prosedur *deployment* yang tidak konsisten antar *operator*; dan (3) ketiadaan mekanisme deteksi otomatis yang dapat mengidentifikasi deviasi sebelum menimbulkan insiden.

Penelitian empiris oleh Zhang et al. (2026) menunjukkan bahwa 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kuberne-

tes mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root* dan tanpa *resource limits*. Temuan tersebut menunjukkan risiko konfigurasi, tetapi defect statis tidak identik dengan drift: penelitian ini membatasi intervensi pada perubahan terhadap acuan yang telah dibekukan.

GitOps menggunakan repositori Git sebagai *source of truth* dan agen rekonsiliasi untuk menyelaraskan keadaan klaster (CNCF GitOps Working Group, 2021). Studi terdahulu telah membahas perbandingan konfigurasi deklaratif dan imperatif, waktu penanganan drift terhadap Ansible, serta rekonsiliasi GitOps dengan pembanding semiotomatis (Nataraja, 2025; Shrestha and Ali, 2024; Matubber, 2025). Kebutuhan penelitian ini adalah bukti yang sesuai dengan konteks InvenioRDM, kepemilikan pengendali dan definisi pengukuran yang transparan, bukan klaim bahwa seluruh perbandingan terkontrol sebelumnya tidak ada.

Perbandingan tetap bermakna untuk mengukur besarnya perbedaan pada kondisi yang dicatat, bukan mengasumsikan GitOps unggul. Tekanan sumber daya, jadwal rekonsiliasi, hooks aplikasi dan pengendali lain dapat memengaruhi respons. Hasil operasional harus dikaitkan dengan konfigurasi serta batas pengamatan yang benar-benar digunakan (Forsgren et al., 2018).

Kesenjangan tersebut menjadi landasan penelitian ini: mengevaluasi secara empiris kesetaraan konfigurasi (R1) serta waktu deteksi, ketersediaan bukti deviasi, dan pemulihan konfigurasi (R2) ketika tujuh skenario penyimpangan terjadi pada dua mode pengelolaan, yaitu imperatif terotomasi dan GitOps, pada satu stack InvenioRDM multi-service. Keterlacakan (R3) tidak diukur karena audit API server Kubernetes tidak tersedia; keterbatasan ini dinyatakan eksplisit. Hasil evaluasi mengkarakterisasi besaran dan kondisi perbedaan pada kasus yang diuji dan menjadi informasi kontekstual, bukan rekomendasi universal pemilihan paradigma.

1.2 Rumusan Masalah

Permasalahan yang dievaluasi adalah ketidaksetaraan konfigurasi terhadap acuan serta keterlambatan sinyal deteksi, ketersediaan bukti deviasi, dan pemulihan konfigurasi (Zhang et al., 2026; Rahman et al., 2023; Pohjola, 2025). Status rollout sehat tidak menjamin konfigurasi cocok, dan riwayat Git tidak otomatis merekam pelaku mutasi di luar Git.

Pembanding pada rancangan ini adalah prosedur imperatif terotomasi tetap berbasis `kubectl`; pemeriksa dan *runbook* otomatis membandingkan keadaan ak-

tual dengan acuan dan menjalankan koreksi tanpa waktu reaksi manusia. Prosedur tersebut bukan *deployment* terpisah: ia adalah mode pengelolaan pada stack InvenioRDM yang sama, dengan *auto-sync* dan *self-heal* ArgoCD dijeda sementara lalu dipulihkan. Mode GitOps menggunakan stack dan acuan yang sama dengan ArgoCD aktif. Kedua mode dijalankan berurutan; tidak ada eksekusi paralel A/B dan tidak ada stack kembar. Kepemilikan HPA normal tetap aktif pada kedua mode; respons replika HPA tidak dapat dijadikan bukti koreksi ArgoCD atas bidang yang diabaikannya.

Evaluasi mencakup dua pertanyaan: (1) bagaimana kesetaraan bidang konfigurasi berbeda setelah intervensi dan koreksi; serta (2) bagaimana waktu deteksi, ketersediaan bukti deviasi, dan waktu pemulihan konfigurasi berbeda. Tujuh skenario meliputi perubahan replika (A), image (B), ConfigMap (C), penghapusan objek (D), patch requests/limits (E), namespace salah (F), dan Secret (G). A dilaporkan sebagai kontrol HPA, F sebagai batas visibilitas objek tak terkelola, dan G sebagai kasus deskriptif yang bukan *self-heal* ArgoCD; ketiganya tidak digabungkan ke estimasi efikasi B–E. Tidak adanya OutOfSync bukan deteksi nol; diff bukan identifikasi akar masalah. R3 tidak diukur karena audit API server tidak tersedia dan tidak dilaporkan sebagai nol atau tidak ada perbedaan.

1.3 Batasan Masalah

Agar penelitian tetap fokus dan terukur, batasan masalah yang ditetapkan adalah:

1. Penelitian dilakukan pada klaster Kubernetes tunggal berbasis RKE2 dengan tiga *node*, yaitu satu *control-plane*/etcd dan dua *worker*, yang menjalankan Kubernetes v1.32.6+rke2r1 pada Ubuntu 24.04 LTS; skenario *multi-cluster* tidak termasuk dalam ruang lingkup.

2. Beban formal adalah satu stack InvenioRDM nyata beserta dependensi multi-service di namespace `invenio`; sasaran sedikitnya 16 komponen memerlukan inventaris terverifikasi. Tidak ada stack kembar `exp-*` dan tidak ada surrogat pada desain formal; S1/S2 historis hanya artefak offline. Karena satu stack dipakai bergantian, beban latar dan dependensi bersama tidak dapat diisolasi dari perlakuan; inferensi dibatasi pada kasus dan apparatus yang benar-benar dicatat.

3. Skenario penyimpangan konfigurasi yang diuji terbatas pada tujuh kategori: (A) perubahan replika, (B) perubahan *image*, (C) modifikasi ConfigMap, (D) *resource deletion*, (E) *patch* manual tidak sah, (F) penyimpangan lintas *namespace*, dan (G) perubahan Secret.

4. Dua mode pengelolaan dibandingkan pada satu stack: (a) mode imperatif terotomasi menggunakan `kubectl` dengan pemeriksa dan *runbook* otomatis, yang diperoleh dengan menjeda *auto-sync*/*self-heal* ArgoCD lalu memulihkannya; dan (b) mode GitOps dengan ArgoCD aktif. HPA normal aktif pada keduanya. Mode dijalankan berurutan; tidak ada eksekusi paralel A/B dan tidak ada *deployment* atau namespace kembar. Perubahan metodologis tetap draf untuk tinjauan pemilik dan pembimbing; eksekusi formal dinonaktifkan sampai seluruh gerbang kesiapan disetujui.

5. Analisis statistik eksploratif menggunakan uji nonparametrik berpasangan/blok bila pemasangan dapat dipertahankan, atau *Mann–Whitney U* dua sisi bila tidak, dengan tingkat signifikansi $\alpha = 0{,}05$ dan ukuran efek (Cohen's d, Cliff's δ, atau Hodges–Lehmann) beserta interval kepercayaan. Waktu deteksi dan pemulihan yang tersensor kanan dianalisis dengan metode *survival* (Kaplan–Meier); data tersensor dilaporkan terpisah dan tidak diimputasi sebagai nol. Observasi berurutan pada stack yang sama tidak diperlakukan independen secara naif.

6. Paradigma perantara (Git sebagai *source of truth* tanpa agen *reconciliation* otomatis) dan paradigma *push-based* (seperti Jenkins CI/CD atau Ansible yang mendorong perubahan ke klaster dari luar) tidak termasuk dalam ruang lingkup. Paradigma perantara tidak memiliki mekanisme koreksi otomatis sehingga tidak memenuhi prinsip GitOps *continuous reconciliation*, sementara paradigma *push-based* memerlukan kredensial klaster yang terekspos dan tidak memiliki *feedback loop* bawaan (Utami and Fatta, 2021; Shrestha and Ali, 2024). Keduanya merepresentasikan titik yang berbeda pada spektrum deklaratif-imperatif dan dapat dievaluasi pada penelitian lanjutan.

7. R3 (bukti keterlacakan) tidak diukur karena audit API server Kubernetes tidak tersedia. Ketiadaan audit tidak menghambat *deployment* atau pengukuran R1/R2; R3 dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai nol atau tidak ada perbedaan.

1.4 Tujuan Penelitian

Tujuan penelitian adalah mengevaluasi perbedaan mode imperatif terotomasi dan GitOps pada kesetaraan konfigurasi (R1) serta waktu deteksi, ketersediaan bukti deviasi, dan pemulihan konfigurasi (R2) dalam konteks satu stack InvenioRDM. R3 tidak diukur karena audit API server tidak tersedia. Hasil tidak diasumsikan mendukung salah satu mode; kontrol HPA, batas visibilitas F, dan kasus deskriptif G tidak digabungkan menjadi klaim efikasi ArgoCD.

1.5 Manfaat Penelitian

Penelitian yang diuraikan dalam proposal ini diharapkan memberikan manfaat berikut:

1. Teoritis: Memberikan bukti empiris terukur mengenai dampak operasional skenario penyimpangan konfigurasi pada dua paradigma *deployment*, yang selama ini banyak didasarkan pada klaim *vendor* atau studi kasus deskriptif. Hasil tersebut melengkapi literatur akademis dengan data eksperimental yang dapat direproduksi dan diverifikasi.

2. Metodologis: Menawarkan protokol eksperimental yang dapat direplikasi, termasuk skenario penyimpangan terstandarisasi, metrik operasional, dan prosedur analisis statistik, sehingga penelitian selanjutnya dapat mereplikasi, memperluas, atau mengkritik temuan pada konteks dan platform yang berbeda.

3. Praktis: Menyediakan bukti empiris terkontrol yang mengkarakterisasi perbedaan operasional antara kedua mode pengelolaan pada skenario penyimpangan yang diuji, serta menyoroti skenario yang paling rentan pada masing-masing mode, sehingga memberi informasi kontekstual bagi praktisi tanpa menjadi rekomendasi universal pemilihan paradigma.

1.6 Sistematika Penulisan

Sistematika penulisan proposal tugas akhir ini adalah sebagai berikut:

- Bab I: Pendahuluan, berisi latar belakang, rumusan masalah, batasan masalah, tujuan penelitian, manfaat penelitian, dan sistematika penulisan.

- Bab II: Penelitian Terkait, membahas studi empiris terkait penyimpangan konfigurasi pada Kubernetes, terminologi dasar, manajemen konfigurasi infrastruktur, paradigma *deployment* alternatif, evaluasi GitOps dan ArgoCD, metode eksperimental pada riset sistem, serta *research gap* yang menjadi landasan penelitian.
- Bab III: Metode dan Rancangan Penelitian, menjelaskan metodologi evaluasi empiris terkontrol, definisi variabel dan intervensi, tujuh skenario penyimpangan, protokol pengukuran, analisis statistik komparatif, ancaman terhadap validitas, dan lingkungan eksperimen.
- Bab IV: Jadwal Penelitian, memuat Gantt Chart berbasis bulan untuk perencanaan waktu penelitian selama 6 bulan.

BAB II

PENELITIAN TERKAIT

2.1 Penyimpangan Konfigurasi (*Configuration Drift*)

Penyimpangan konfigurasi (*configuration drift*) adalah situasi di mana *state* aktual sebuah sistem menyimpang dari *state* yang didefinisikan secara deklaratif (Rahman et al., 2020; Hassan et al., 2024). Dalam konteks Kubernetes, penyimpangan terjadi ketika *operator*, baik secara langsung maupun tidak sengaja, mengubah manifest klaster tanpa memperbarui *source of truth* deklaratif, sehingga *state* aktual dan *desired state* menjadi tidak konsisten.

Penelitian empiris oleh Zhang et al. (2026) melakukan studi terhadap 719 *defect* konfigurasi dari 2.260 skrip Kubernetes dan mengidentifikasi 15 kategori *defect*, di mana 533 dari 719 *defect* (74,1%) menyebabkan *crash*, operasi yang salah, atau *outage*. Studi tersebut menegaskan bahwa *misconfiguration* merupakan risiko operasional utama pada Kubernetes. Rahman et al. (2023) menambahkan bahwa *security misconfiguration* pada manifest Kubernetes *open-source* mencakup 11 kategori yang berulang, termasuk *container* yang berjalan sebagai *root*, tanpa *drop capabilities*, dan tanpa *resource limits*. Temuan tersebut didukung oleh Rahman et al. (2019) yang mengidentifikasi tujuh *security smells* berulang pada skrip IaC melalui studi empiris terhadap 1.726 skrip dan 15.232 skrip dari 293 repositori *open-source*. Penelitian menunjukkan bahwa penyimpangan konfigurasi bukan hanya masalah estetika konfigurasi, tetapi ancaman nyata terhadap keandalan dan keamanan layanan.

Rahman et al. (2020) memberikan taksonomi empiris untuk memahami sumber dan dampak penyimpangan pada sistem *Infrastructure-as-Code* (IaC). Melalui analisis kualitatif terhadap 1.448 *commit* terkait *defect* dan kuantifikasi terhadap 80.425 *commit* dari 291 repositori, penelitian tersebut mengidentifikasi delapan kategori *defect*, di mana kategori data konfigurasi yang keliru merupakan kategori paling sering muncul dan kategori *idempotency* merupakan kategori khas IaC yang paling disepakati oleh 66 praktisi yang disurvei. Taksonomi tersebut menyediakan dasar empiris untuk mengoperasionalkan faktor pendorong penyimpangan pada R1, menggantikan kerangka konseptual tanpa validasi Kubernetes-eksklusif.

2.1.1 Faktor yang Menimbulkan Penyimpangan Konfigurasi

Berdasarkan literatur, faktor-faktor utama yang memperburuk penyimpangan konfigurasi pada Kubernetes meliputi:

1. Perubahan langsung pada klaster: Penggunaan `kubectl edit`, `kubectl patch`, atau `kubectl scale` secara langsung tanpa memperbarui repositori *source of truth* (Zhang et al., 2026).

2. Prosedur *deployment* yang tidak terdokumentasi: Setiap *operator* dapat memiliki urutan perintah yang berbeda, yang mengakibatkan *state* klaster tidak deterministik (Rahman et al., 2020).

3. Kurangnya mekanisme *rollback* yang terintegrasi: *Deployment* manual seringkali tidak memiliki mekanisme *rollback* otomatis; *operator* harus mengingat konfigurasi sebelumnya atau mengandalkan *backup* yang mungkin tidak konsisten (Rahman et al., 2023, 2019).

4. Ketiadaan mekanisme deteksi penyimpangan otomatis: Tanpa pemeriksaan berkala antara *state* aktual dan *desired state*, penyimpangan dapat terakumulasi selama periode lama tanpa terdeteksi (Zhang et al., 2026).

2.2 Terminologi Dasar

Bagian ini mendefinisikan terminologi kunci yang digunakan dalam penelitian, dengan tujuan membangun dasar konseptual yang konsisten sebelum pembahasan lebih lanjut.

2.2.1 Kubernetes dan Orkestrasi Kontainer

Kubernetes adalah platform *container orchestration open-source* yang mengotomasi *deployment*, *scaling*, dan manajemen aplikasi terkemas (*containerized*) (Burns et al., 2016). Kubernetes awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem *workload* berskala besar di Borg dan Omega, dan kini menjadi standar de facto untuk orkestrasi kontainer di industri (Verma et al., 2015). Pada Kubernetes, *state* yang diinginkan (*desired state*) dideklarasikan melalui berkas konfigurasi (*manifest*), dan *control plane* bertanggung jawab mencapai dan mempertahankan *state* tersebut.

2.2.2 Git dan *Version Control*

Git adalah sistem *version control* terdistribusi yang merekam setiap perubahan pada berkas beserta metadata (penulis, *timestamp*, pesan *commit*) (Chacon and Straub, 2014). Dalam konteks GitOps, Git berfungsi sebagai *single source of truth* yang menyimpan deklarasi *state* infrastruktur dan aplikasi, sehingga setiap perubahan dapat dilacak, di-*review*, dan dikembalikan (*rollback*) secara andal.

2.2.3 *Continuous Integration* dan *Continuous Deployment*

Continuous Integration (CI) adalah praktik di mana perubahan kode secara otomatis diuji dan digabungkan ke *branch* utama, sementara *Continuous Deployment* (CD) adalah perluasan CI di mana perubahan yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Humble and Farley, 2010; Ebert et al., 2016). GitOps merupakan paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth*.

2.2.4 *Infrastructure-as-Code* (IaC)

Infrastructure-as-Code (IaC) adalah praktik mendefinisikan dan mengelola infrastruktur melalui berkas konfigurasi yang diberi versi (*versioned*), bukan melalui proses manual (Morris, 2020). IaC memungkinkan *version control*, *code review*, dan reprodusibilitas lingkungan. Dalam konteks Kubernetes, IaC diimplementasikan melalui *manifest* deklaratif (YAML) yang mendefinisikan *desired state* klaster.

2.2.5 Paradigma Deklaratif vs Imperatif

Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (*how to achieve the state*). Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* dan sistem bertanggung jawab mencapai serta mempertahankan *state* tersebut (*what state to achieve*) (Scott, 2015). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual eksplisit untuk setiap deviasi.

2.3 Manajemen Konfigurasi Infrastruktur

Infrastructure state management adalah disiplin yang mengatur bagaimana *state* sebuah sistem infrastruktur didefinisikan, diterapkan, dan dipertahankan. Dalam paradigma deklaratif, *operator* mendefinisikan *desired state* melalui konfigurasi yang telah dibakukan (*Infrastructure-as-Code*), dan sistem bertanggung jawab untuk mencapai serta mempertahankan *state* tersebut (Morris, 2020). Dalam paradigma imperatif, *operator* memberikan serangkaian perintah langkah demi langkah yang mengubah *state* sistem secara langsung (Limoncelli et al., 2018). Perbedaan fundamental ini memiliki implikasi langsung terhadap bagaimana sistem merespons deviasi konfigurasi: paradigma deklaratif mendukung *convergence* otomatis, sementara paradigma imperatif memerlukan intervensi manual untuk setiap deviasi.

2.4 *Continuous Deployment* dan Paradigma GitOps

Continuous Deployment (CD) adalah praktik dalam ranah DevOps di mana setiap perubahan kode yang lulus pengujian secara otomatis di-*deploy* ke lingkungan produksi tanpa intervensi manual eksplisit (Limoncelli et al., 2018; Ebert et al., 2016). CD merupakan perluasan dari *Continuous Integration* (CI) dan bertujuan meminimalkan waktu antara penulisan kode dan ketersediaannya bagi pengguna akhir. Dalam ekosistem Kubernetes, CD memerlukan mekanisme yang dapat secara andal menyelaraskan perubahan konfigurasi dengan *state* klaster.

GitOps adalah paradigma spesifik di dalam CD yang menggunakan repositori Git sebagai *single source of truth* untuk mendefinisikan *state* infrastruktur dan aplikasi. CNCF GitOps Working Group mendefinisikan empat prinsip OpenGitOps: (1) sistem dideklarasikan secara deklaratif; (2) *state* sistem yang diinginkan tersimpan dan diberi versi (*versioned*) dalam Git; (3) perubahan *state* ditarik (*pulled*) secara otomatis oleh agen *software*; dan (4) agen *software* secara kontinu mendeteksi dan mengoreksi deviasi dari *state* yang diinginkan (CNCF GitOps Working Group, 2021).

Karakteristik GitOps yang berbeda dari pendekatan imperatif tradisional meliputi: *audit trail* lengkap melalui Git *history*, penerapan *code review* pada perubahan infrastruktur, *rollback* yang sederhana melalui `git revert`, dan reprodusibilitas konfigurasi di berbagai lingkungan (Yuen et al., 2021). Dalam model *pull-based* GitOps, agen yang berjalan di dalam klaster secara periodik membandingkan *state* Git dengan *state* klaster dan menyelaraskan keduanya, menghilangkan kebutuhan akan

akses kredensial klaster dari luar (Utami and Fatta, 2021). Klaim adopsi dan manfaat GitOps juga muncul dalam artikel industri CNCF 2025; artikel tersebut dipakai hanya sebagai konteks industri, bukan sebagai bukti *peer-review* (Lopez and Narang, 2025).

2.4.1 Performa *Deployment* dan *Anti-Pattern*

Forsgren dkk. (Forsgren et al., 2018) mengidentifikasi empat metrik kunci yang mengkarakterisasi performa *deployment* pada organisasi teknologi: (1) *deployment frequency*, (2) *lead time for changes*, (3) *time to restore service*, dan (4) *change failure rate*. Operasionalisasi keempat metrik tersebut untuk pengukuran otomatis dibahas oleh Sallin et al. (2021) melalui prototipe pengukuran otomatis pada XP 2021, sementara Wilkes et al. (2023) memformalkan kerangka pengukuran DORA yang independen terhadap siklus hidup pengembangan dan memvalidasinya pada 304 repositori *open-source*, sehingga mendukung justifikasi pengumpulan metrik berbasis *polling* dalam penelitian ini. Metrik-metrik tersebut menunjukkan bahwa organisasi yang mengadopsi praktik *continuous deployment* dan *Infrastructure-as-Code* secara konsisten mengungguli organisasi yang bergantung pada proses manual. Lebih lanjut, praktik *deployment* manual yang mengabaikan *version control* dan *automated reconciliation* merupakan *anti-pattern* yang berkontribusi terhadap penyimpangan konfigurasi (Beyer et al., 2016). Dalam konteks penelitian yang diuraikan dalam proposal ini, metrik *time to restore service* dan *change failure rate* berelasi langsung dengan metrik R2 (waktu deteksi/pemulihan) dan R1 (konsistensi *deployment*) yang digunakan dalam eksperimen.

ArgoCD mengimplementasikan prinsip-prinsip GitOps melalui *reconciliation loop* yang berjalan di dalam klaster. ArgoCD terdiri dari tiga komponen utama: (1) *application controller* yang secara periodik membandingkan *state* Git dengan *state* aktual dan melakukan tindakan korektif; (2) *repo server* yang mengelola koneksi ke repositori Git; dan (3) *Redis cache* yang menyimpan *state* sementara untuk performa (Argo Project, 2024). ArgoCD berperan sebagai *negative feedback controller*: setiap deviasi (penyimpangan) bersifat *transien*, yaitu sistem secara otomatis kembali ke *desired state* tanpa memerlukan intervensi manusia, sepanjang *self-heal* diaktifkan.

2.5 Dasar Teori: *Reconciliation* dan *Convergence* pada Sistem Deklaratif

Karakteristik GitOps yang berbeda dari pendekatan imperatif dapat dijelaskan melalui dua prinsip teoretis yang saling berkaitan.

2.5.1 *Convergence* Menuju *Desired State*

Konsep *negative feedback controller* menjelaskan upaya sistem mencapai keadaan yang diinginkan (Limoncelli et al., 2018). ArgoCD membandingkan deklarasi dengan keadaan klaster dan dapat mengoreksi perbedaan sesuai kebijakan, izin dan cakupannya (Argo Project, 2024; Hassan et al., 2024). Hal ini tidak menjamin setiap perubahan ditemukan atau pulih: bidang yang diabaikan, objek tak terkelola, kegagalan pengendali dan kepemilikan HPA membatasi interpretasi. Pada rancangan ini, respons replika HPA dipisahkan dari self-heal ArgoCD.

Perintah imperatif tidak dengan sendirinya menyediakan rekonsiliasi berkelanjutan, tetapi pemeriksaan dan koreksinya dapat diotomasi (Hassan et al., 2024; Rahman et al., 2020). Pembanding penelitian ini menggunakan prosedur imperatif terotomasi tetap; pemeriksa dan runbook berjalan otomatis tanpa waktu reaksi manusia. Waktu reaksi manusia tidak dimasukkan sebagai variabel.

2.5.2 *Audit Trail* sebagai *Social Control Mechanism*

Git merekam perubahan deklarasi yang dikomit beserta metadata dan dapat mendukung peninjauan perubahan (CNCF GitOps Working Group, 2021). Mutasi langsung pada klaster tidak otomatis masuk riwayat Git. Bukti audit API dan tindakan pengendali tetap diperlukan untuk merekonstruksi pelaku, urutan, dan hasil mutasi. Namun, R3 tidak diukur dalam studi ini karena audit API server Kubernetes tidak tersedia; keterlacakan dibahas sebagai keterbatasan dan pekerjaan lanjutan, bukan sebagai hasil.

2.6 *Runbook* sebagai Standarisasi Prosedur Pemulihan

Runbook adalah dokumen terstandarisasi yang berisi prosedur operasional untuk menanggapi insiden atau melakukan pemulihan (*recovery*) pada sebuah sistem. Dalam ranah operasi TI, *runbook* berfungsi sebagai *playbook* yang mendefinisikan langkah-langkah korektif berurutan berdasarkan jenis insiden, sehingga setiap *operator* yang berbeda mengikuti urutan tindakan yang sama (Schlette et al., 2024). Penggunaan *runbook* mengurangi variabilitas akibat keahlian *operator* yang berbeda-beda dan memastikan bahwa keputusan subjektif diminimalkan selama pemulihan.

Pada rancangan ini, runbook merupakan prosedur koreksi otomatis berdasarkan perbedaan yang teramati, tanpa menerima label skenario dari injector. Entry point

`recovery-runbook.sh` saat ini memvalidasi bukti hasil koreksi dan reset secara offline. Eksekusi langsung tetap dinonaktifkan sampai integrasi lifecycle dan postcondition disetujui. Selesainya diagnosis atau rollout sehat bukan bukti seluruh konfigurasi maupun data aplikasi telah pulih.

2.7 Paradigma *Deployment* Alternatif

Selain paradigma manual dan GitOps yang menjadi fokus penelitian yang diuraikan dalam proposal ini, terdapat paradigma *deployment* lain yang menempati posisi berbeda pada spektrum deklaratif-imperatif. Pertama, paradigma *push-based Continuous Deployment* (misalnya Jenkins CI/CD) mendorong perubahan dari luar klaster ke dalam klaster melalui *pipeline* otomatis. Meskipun menawarkan otomasi *deployment*, paradigma ini memerlukan kredensial klaster yang terekspos pada *pipeline* eksternal dan tidak memiliki mekanisme *reconciliation* bawaan: ketika *state* klaster menyimpang, *pipeline* tidak secara aktif mendeteksi atau mengoreksi deviasi (Utami and Fatta, 2021). Kedua, manajemen konfigurasi imperatif menggunakan alat seperti Ansible atau Terraform menerapkan perubahan melalui perintah imperatif yang dijalankan dari luar klaster. Shrestha and Ali (2024) mengevaluasi Ansible sebagai alternatif GitOps dan menemukan bahwa meskipun Ansible menyediakan otomasi *deployment*, ia tidak memiliki *continuous reconciliation* dan *audit trail* bawaan yang dimiliki GitOps. Ketiga, paradigma perantara — Git sebagai *source of truth* tanpa agen *reconciliation* otomatis — merepresentasikan titik di mana deklarasi *state* disimpan dalam Git, tetapi koreksi penyimpangan tetap bergantung pada intervensi manual.

Perbandingan difokuskan pada prosedur imperatif terotomasi terstandarisasi dan ArgoCD dengan konfigurasi yang dicatat. Tidak ada klaim bahwa selisihnya menjadi batas atas seluruh alternatif. HPA normal pada kedua mode merupakan pengendali bersama, bukan perbedaan paradigma.

2.8 Evaluasi Empiris GitOps dan ArgoCD

Utami and Fatta (2021) melakukan analisis perbandingan model *deployment* deklaratif *pull-based* (GitOps dengan ArgoCD) versus *push-based* pada Kubernetes, dan menemukan bahwa model *pull-based* dilaporkan lebih andal karena agen berjalan di dalam klaster dan tidak memerlukan kredensial eksternal. Shrestha and Ali (2024)

mengevaluasi GitOps versus Ansible untuk manajemen konfigurasi Kubernetes, termasuk waktu induksi, penemuan, dan penanganan drift; pembanding Ansible dan konteks protokolnya berbeda dari prosedur imperatif tetap serta kepemilikan pengendali yang dirancang di sini. Nataraja (2025) melakukan analisis berpusat keamanan terhadap pendekatan deklaratif (GitOps/ArgoCD) dan imperatif (Jenkins/`kubectl`) pada Minikube satu node dengan konfigurasi keamanan yang berbeda, dan melaporkan koreksi penyimpangan yang lebih cepat, *compliance rate* yang lebih tinggi, serta paparan kerentanan yang lebih rendah. Skala satu node dan perbedaan konfigurasi tersebut membatasi transfer temuan ke klaster multi-node dan apparatus penelitian ini.

Di sisi fondasi empiris IaC, Hassan et al. (2024) mengkategorikan 5.110 *defect* rekonsiliasi *state* ke dalam delapan kategori sebagai landasan teori *reconciliation loop*, sementara Rahman and Williams (2019) mengidentifikasi 10 properti kode sumber yang berkorelasi dengan skrip IaC yang cacat dan mendukung pemilihan uji *Mann–Whitney* beserta ukuran efek Cliff's Delta. Matubber (2025) mengukur *reconciliation time*, *scalability*, dan *reliability* GitOps untuk infrastruktur K8s, termasuk *self-healing* terhadap penyimpangan konfigurasi; pembandingnya berupa alur kerja lama yang semi-terotomasi, bukan prosedur imperatif tetap dengan acuan dan kepemilikan pengendali yang sama seperti rancangan ini.

Tabel 2.1 merangkum kelemahan metodologis kunci dari penelitian-penelitian tersebut.

Tabel 2.1: Perbandingan Kelemahan Metodologis Penelitian Terdahulu

Penelitian	Fokus	Kelemahan Metodologis
Shrestha and Ali (2024)	GitOps vs Ansible	Penilaian kualitatif untuk tingkat keparahan penyimpangan; rincian uji formal tidak dinilai dalam ringkasan ini
Nataraja (2025)	Keamanan deklaratif vs imperatif	Minikube satu node dan konfigurasi keamanan berbeda membatasi transfer konteks
Matubber (2025)	*Reconciliation time* GitOps	Pembanding legacy semiotomatis bukan prosedur imperatif tetap penelitian ini

2.9 Metode Eksperimental Terkontrol pada Riset Sistem

Evaluasi empiris pada riset sistem dan infrastruktur memerlukan desain eksperimental yang memitigasi ancaman terhadap validitas (Wohlin et al., 2012). Limoncelli et al. (2018) menjelaskan bahwa penelitian sistem harus membangun lingkungan yang dapat direproduksi, mengendalikan variabel pengganggu, dan mendefinisikan metrik yang jelas sebelum menyimpulkan hubungan kausal.

Rancangan membandingkan dua prosedur dengan intervensi dan acuan setara serta kendali pengganggu yang didokumentasikan (Wohlin et al., 2012). Pengendalian tersebut tidak menjamin paradigma abstrak menjadi satu-satunya perbedaan; jadwal, implementasi pengendali dan kepemilikan bidang tetap membatasi inferensi.

Analisis eksploratif dapat menggunakan Mann–Whitney U dan ukuran efek untuk waktu teramati lengkap dengan asumsi yang sesuai (Wohlin et al., 2012; Rahman and Williams, 2019). Data tersensor tidak boleh diubah menjadi waktu sukses atau diam-diam dibuang; aturan analisis dan target pengulangan dijelaskan pada Bab III. A dan F tetap terpisah dari efikasi ArgoCD.

2.10 Klasifikasi Beban Uji (*Workload*)

Aplikasi yang dijalankan pada klaster Kubernetes dapat dikelompokkan ke dalam beberapa kelas *workload* berdasarkan karakteristik arsitektur dan manajemen *state* (Siddiqui et al., 2023; Aghili et al., 2024):

1. *Workload* web tanpa state (*stateless web microservice*): Layanan yang tidak menyimpan *state* secara internal, sehingga setiap permintaan dapat dilayani oleh instans mana pun. Contoh: server HTTP sederhana, *API gateway*.

2. *Workload* berbasis data yang menyimpan *state* (*stateful data service*): Layanan yang menyimpan *state* secara persisten, seperti *database* relasional, *search engine*, atau *object storage*. *Workload* kelas ini memerlukan urutan *deployment* dan penanganan penyimpangan yang lebih kompleks.

3. *Workload* komposit multi-service: Aplikasi yang terdiri atas banyak komponen yang saling bergantung, mencakup kombinasi kelas 1 dan 2 beserta komponen infrastruktur (*ingress*, TLS, *secrets*, *monitoring*, *backup*, kebijakan keamanan). Kelas *workload* ini memiliki kompleksitas dependensi tertinggi dan paling

rentan terhadap penyimpangan konfigurasi akibat banyaknya komponen yang harus diselaraskan.

Beban formal penelitian adalah InvenioRDM nyata, meliputi web, worker/scheduler, PostgreSQL, OpenSearch, Redis, penyimpanan persisten dan infrastruktur pendukung. Sasaran cakupan sedikitnya 16 komponen memerlukan inventaris dari sumber deployment dan keadaan aktual; daftar konseptual bukan bukti jumlah komponen atau penggunaan MinIO/S3 oleh aplikasi. Beban uji merupakan apparatus studi empiris, bukan produk yang dikembangkan. Temuan dibatasi pada konfigurasi InvenioRDM yang benar-benar digunakan; NGINX S1/S2 historis tidak menjadi pengganti atau dasar generalisasi formal.

2.11 Kesenjangan Penelitian (*Research Gap*)

Berdasarkan tinjauan pustaka yang telah diuraikan, terdapat empat kesenjangan yang saling berkaitan dan menjadi landasan penelitian:

1. Kesenjangan Validitas Konstruk: Studi terdahulu memakai definisi penyimpangan, metrik, dan cakupan yang berbeda. Shrestha and Ali (2024) memakai penilaian kualitatif untuk tingkat keparahan, sedangkan Nataraja (2025) berfokus pada keamanan dan *compliance rate*. Perbedaan operasionalisasi tersebut membatasi perbandingan langsung dengan kesetaraan bidang, kesehatan, dan bukti deviasi yang didefinisikan dalam rancangan ini.

2. Kesenjangan Validitas Internal: Konfigurasi apparatus, skala klaster, pembanding, dan kepemilikan pengendali berbeda antarstudi. Nataraja (2025) memakai Minikube satu node dengan konfigurasi keamanan berbeda, sedangkan Matubber (2025) memakai pembanding legacy semiotomatis. Perbedaan tersebut membatasi transfer atribusi kausal ke konteks InvenioRDM dengan HPA normal pada kedua mode.

3. Kesenjangan Metodologis: Belum ditemukan evaluasi yang menggunakan kontrak bidang konfigurasi yang sama pada kedua mode, memisahkan sinyal pengendali dari rollout, serta mempertahankan hasil tersensor dan kegagalan reset pada beban InvenioRDM ini. Rancangan ini ditujukan untuk mengisi kebutuhan tersebut; klaim mengenai hasilnya menunggu data formal.

4. Kesenjangan Kontekstual: Evaluasi GitOps yang ada dilakukan pada konteks dan protokol yang bervariasi. Belum ditemukan evaluasi terkontrol pada apparatus InvenioRDM nyata dengan dependensi multi-service, kepemilikan HPA, serta pemisahan skenario kontrol, drift terkelola, dan batas visibilitas seperti yang dirancang di sini.

Rancangan ini mengoperasionalkan R1 dan R2 dalam konteks tersebut, dengan pengendalian sumber serta pengukuran yang dapat diperiksa. Kontribusi yang diklaim bersifat terbatas dan dapat diperiksa: (1) perbandingan empiris pada *deployment* InvenioRDM multi-service nyata, bukan *workload* sintetis; (2) dua mode yang dijalankan berurutan pada satu stack produksi-like, dengan mode imperatif didefinisikan sebagai penjedaan ArgoCD ditambah pemeriksa/*runbook* `kubectl`, sehingga perbandingan tidak memerlukan stack kembar; (3) pengukuran R1/R2 yang dapat direproduksi dengan kepemilikan HPA, batas visibilitas F, dan kasus G yang dipisahkan dari efikasi; serta (4) bukti batas bahwa hasilnya berlaku untuk kasus dan apparatus yang diuji, bukan klaim superioritas GitOps universal. Besarnya perbedaan baru dapat dinilai setelah metode ditinjau, apparatus divalidasi, dan data formal dikumpulkan.

BAB III
METODE DAN RANCANGAN PENELITIAN

3.1 Deskripsi Umum dan Status Rancangan

Penelitian ini merupakan studi kasus komparatif terkontrol (*controlled single-case comparative study*) atas dua mode pengelolaan pada satu stack InvenioRDM produksi-like: (1) mode imperatif terotomasi berbasis `kubectl` dan (2) mode GitOps berbasis ArgoCD. Kedua mode bukan dua *deployment* terpisah dan bukan dua stack kembar. Pada mode imperatif, *auto-sync* dan *self-heal* ArgoCD dijeda, sedangkan pemeriksa dan *runbook* otomatis tetap membandingkan keadaan aktual dengan acuan serta menjalankan koreksi `kubectl`; setelah blok selesai, ArgoCD dipulihkan dan diverifikasi. Pada mode GitOps, ArgoCD aktif melakukan rekonsiliasi. Tidak ada operator yang menunggu, mengetik, atau memilih tindakan korektif selama pengukuran; waktu reaksi manusia bukan variabel penelitian.

Kedua mode dijalankan berurutan pada stack yang sama. Tidak ada eksekusi paralel A/B dan tidak ada namespace `exp-*` pengganti. Konsekuensinya, penelitian ini tidak mengisolasi perlakuan dari beban latar, dependensi bersama, atau keadaan klaster yang berubah; non-isolasi tersebut merupakan keterbatasan eksplisit, bukan variabel yang dikendalikan. Yang dibandingkan adalah dua prosedur operasional pada satu kasus, bukan keunggulan universal suatu paradigma.

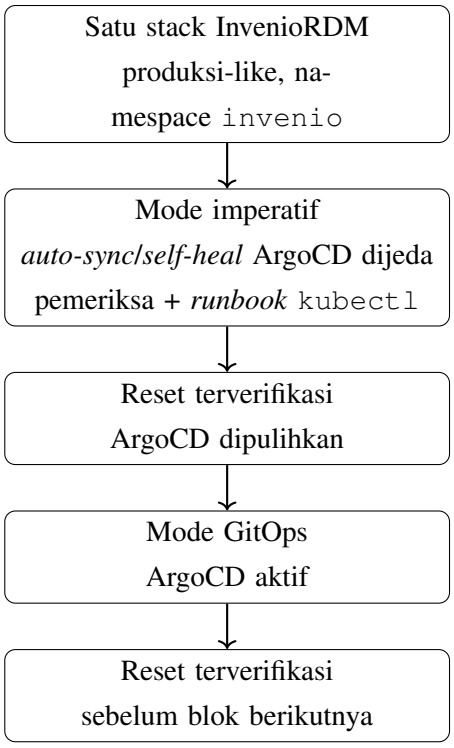
Revisi metodologis ini merupakan draf untuk tinjauan pemilik penelitian dan pembimbing. Eksperimen formal belum disetujui dan eksekusi langsung instrumen dinonaktifkan. Kelulusan uji offline tidak membuktikan kesiapan klaster, SMTP, kapasitas, ataupun pemulihan data aplikasi. Protokol imperatif terotomasi dan kolektor pasif belum diuji secara formal pada klaster; pilot terbatas pada stack yang sama bukan data formal. Proposal yang diterima sebelumnya dan artefak historis tetap dapat ditelusuri melalui riwayat versi; perubahan definisi tidak mengubah makna hasil historis menjadi hasil formal.

3.2 Metodologi dan Kesetaraan Perbandingan

Rancangan menggabungkan elemen eksperimen terkontrol dengan studi kasus tunggal: kondisi dasar diukur, satu intervensi dilakukan, dan kondisi pascaintervensi diamati pada kasus yang sama (Wohlin et al., 2012; Runeson and Höst, 2009). Kare-

na kasus tidak dipilih acak dari populasi dan kedua mode berurutan pada stack yang sama, rancangan tidak memberikan inferensi populasi (Cook and Campbell, 1979). Unit analisis adalah satu injeksi yang diterima pada satu mode dalam satu pengulangan.

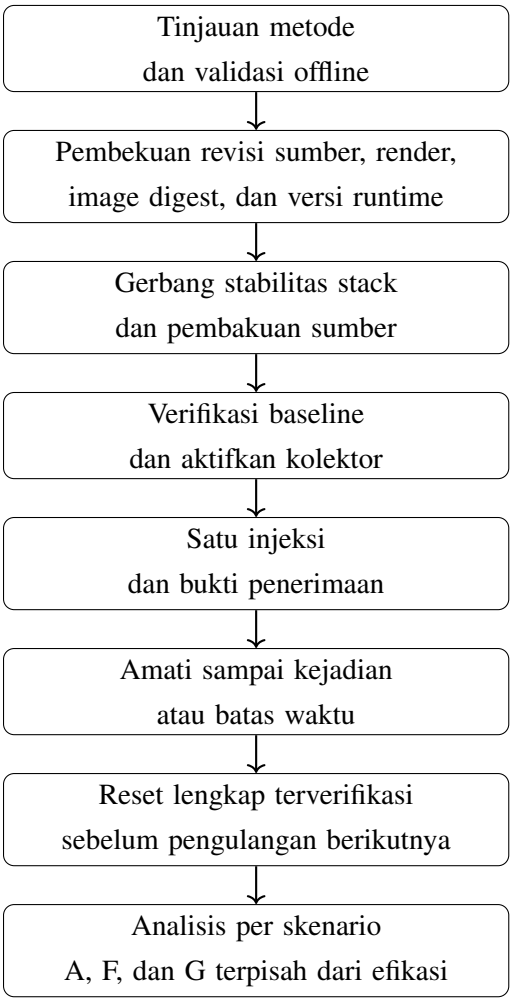
Kedua mode menggunakan sumber *deployment*, image digest, inventaris, kontrak bidang konfigurasi, kebijakan HPA, dan aturan pengukuran yang setara. Pada mode imperatif, kepemilikan rekonsiliasi ArgoCD dijeda secara terkendali: *auto-sync* dan *self-heal* pada `apps` dan `invenio-bootstrap` dinonaktifkan sementara, lalu dipulihkan pada blok `finally` dan diverifikasi (Argo Project, 2025). Pemeriksa imperatif hanya menerima keadaan aktual dan acuan; ia tidak menerima identitas skenario. Pada mode GitOps, ArgoCD aktif dan menjadi mekanisme rekonsiliasi. Urutan mode diacak dan dicatat; urutan skenario memakai permutasi yang sama di kedua mode. Setiap run dimulai dari baseline yang terverifikasi dan hanya satu injeksi diperbolehkan setelah reset sebelumnya terverifikasi.



Gambar 3.1: Dua mode pengelolaan dijalankan berurutan pada satu stack InvenioRDM; mode imperatif adalah penjedaan ArgoCD, bukan *deployment* terpisah.

Gambar 3.1 menunjukkan bahwa hanya satu mode aktif pada satu waktu dan

setiap perpindahan mode didahului reset serta pemulihan ArgoCD yang terverifikasi. HPA merupakan pengendali bersama, sehingga responsnya tidak diatribusikan kepada ArgoCD maupun kepada pemeriksa imperatif.



Gambar 3.2: Bagan alir rancangan penelitian dengan satu mode aktif pada satu waktu; pelaksanaan formal tetap menunggu persetujuan.

Gambar 3.2 menempatkan pembekuan sumber dan gerbang kesiapan sebelum injeksi serta reset lengkap sebelum pengulangan berikutnya. Perpindahan mode mengikuti urutan yang diacak dan dicatat. Kegagalan dan data tersensor disimpan, bukan dihapus agar hanya pemulihan sukses yang dilaporkan.

3.3 Lingkungan dan Provenans Deployment

Konteks lingkungan yang terverifikasi pada 25 Oktober 2026 adalah sebagai berikut:

- Tiga node RKE2 berstatus *Ready*: satu *control-plane*/etcd dan dua *worker*.
- Kubernetes v1.32.6+rke2r1 pada Ubuntu 24.04 LTS.
- 18 dari 18 Application ArgoCD berstatus *Synced/Healthy*; seluruhnya memakai *automated sync* dan *self-heal*, sedangkan `argocd-self` dan `sealed-secrets` memakai `prune: false`. Pada mode imperatif, *auto-sync*/*self-heal* dijeda sementara dan dipulihkan setelah blok.
- Satu stack aplikasi InvenioRDM di namespace `invenio`; tidak ada namespace kembar `exp-*`. Namespace pendukung yang tercatat meliputi `argocd`, `database`, `monitoring`, dan `velero`.
- Beban aktif mencakup web 2 replika (HPA 2–4), worker 3 replika (HPA 1–4), scheduler 1 replika, PostgreSQL 1 instance CNPG tanpa failover dengan backup harian pukul 02.00 UTC dan retensi 30 hari, Redis master, OpenSearch satu node, MinIO, Loki/Grafana, dan Velero *node-agent*.
- MinIO berperan sebagai *primary file store* sekaligus *backup store* pada NFS yang sama; direktori `/export` berisi sekitar 35 GiB pada snapshot tersebut, PVC meminta 20 GiB, dan tidak ada salinan independen di luar domain kegagalan tersebut.
- Kapasitas yang terverifikasi hanya untuk satu stack InvenioRDM penuh. Sampel 25 Oktober 2026 menunjukkan *worker-01* dan *worker-02* masing-masing memakai 59% memori, tetapi belum ada jendela pengamatan berkelanjutan atau bukti penempatan dua stack.
- *Seed* saat ini berupa draf privat `4ben9-bgh38` dengan berkas `seed.bin` berukuran 65.536 byte; unggah dan unduh publiknya terverifikasi, dan belum ada record publik. Nilai MD5/SHA-256 serta bukti resetnya tercatat pada snapshot dan tidak diulang di sini.

Versi ArgoCD v2.12.0, OpenSearch 2.19.1, dan MinIO `RELEASE.2024-12-18T13-15-44Z` tercatat pada snapshot 25 Oktober

2026; versi CNPG belum dinyatakan dan seluruh versi runtime harus direkam ulang sebelum pengumpulan formal.

Stack formal adalah stack InvenioRDM yang sama di namespace `invenio`. Tidak ada salinan namespace `exp-*`, tidak ada *deployment* kembar, dan tidak ada eksekusi paralel. Injeksi dan koreksi dibatasi pada objek yang tercantum dalam kontrak bidang; objek lain pada namespace tersebut tidak disentuh. Karena tidak ada twin, keadaan cluster-wide dan dependensi bersama tidak dapat diisolasi dari perlakuan. Pilot 27 Oktober 2026 memakai stack yang sama dan merupakan bukti pilot, bukan data formal.

Sumber otoritatif adalah repositori `https://github.com/vityasyyy/invenio-rdm-gitops`. Revisi sumber yang diperiksa untuk kontrak bidang ialah `6cacea6642a7c1690243d3b54130fa08216a9595`; penunjuknya disimpan pada `experiments/deployment-source.json`. Snapshot lingkungan di atas diverifikasi terhadap `main` pada `d7ef120`. Application `argocd/apps/invenio-bootstrap.yaml` memilih `k8s/apps/invenio/kustomization.yaml`, yang merujuk sumber daya dan image digest. Kedua revisi adalah referensi, bukan pernyataan bahwa klaster sedang menjalankan revisi itu. Tidak dibuat salinan manifest Invenio editable di repositori penelitian.

Sebelum eksperimen, catat revisi sumber aktual, hasil render beserta SHA-256, image digest, versi ArgoCD dan chart, inventaris objek, serta kepemilikan pengendali. Hasil render merupakan artefak bukti tak berubah, bukan sumber konfigurasi kedua. Dependensi bersama dibedakan dari sumber daya yang diisolasi agar reset satu mode tidak mengubah data mode lain.

Konfigurasi sumber `k8s/infra/argocd/patches/argocd-cm.yaml` menetapkan `timeout.reconciliation` sebesar 30 detik, berbeda dari target proposal lama 60 detik. Interval rekonsiliasi repositori ini tidak sama dengan watch kejadian Kubernetes, retry self-heal, atau interval evaluator. Seluruh interval dan perilaku aktual harus dicatat serta dibekukan sebelum formal, bukan disesuaikan setelah melihat hasil.

3.4 Intervensi dan Kepemilikan Pengendali

Pemilihan kategori mempertimbangkan defect konfigurasi Kubernetes dan praktik operasional (Zhang et al., 2026; Pohjola, 2025). Tabel 3.1 membedakan kontrol HPA, drift terkelola, batas visibilitas, dan kasus deskriptif.

Tabel 3.1: Matriks skenario dan batas interpretasi

Skenario	Intervensi	Kepemilikan dan analisis
A	Perubahan replika Deployment	HPA adalah pemilik normal replika pada kedua mode; `/spec/replicas` dikecualikan dari R1 statis; pemulihan replika oleh HPA bukan bukti ArgoCD maupun koreksi imperatif
B	Perubahan image	Bidang Deployment terpilih; image digest acuan dibekukan; drift terkelola
C	Modifikasi ConfigMap	Kesetaraan objek dan adopsi konfigurasi proses dibedakan; drift terkelola
D	Penghapusan sumber daya	Verifikasi penciptaan ulang dan seluruh bidang; bukan sekadar rollout sehat; drift terkelola
E	Patch requests/limits	Seluruh bidang terpilih dibandingkan dengan aturan identik; drift terkelola
F	Objek pada namespace salah	Batas visibilitas objek tak terkelola; tidak diasumsikan dipangkas ArgoCD; deskriptif
G	Anotasi metadata Secret sintetis	Bukan kasus *self-heal* ArgoCD; pemulihan dilakukan harness; deskriptif dan tidak dipool

Pada skenario A, HPA web dan worker tetap memiliki replika pada kedua mode. HPA adalah pemilik normal jumlah replika; perubahan `/spec/replicas` oleh injeksi dapat dikembalikan oleh HPA, bukan oleh ArgoCD atau oleh pemeriksa imperatif. Karena itu `/spec/replicas` dikecualikan dari R1 statis pada Deployment yang dimiliki HPA, dan nilainya dilaporkan sebagai status HPA terpisah. Tidak munculnya *OutOfSync* bukan waktu deteksi nol, dan kembalinya replika bukan bukti *self-heal* ArgoCD. Nilai acuan dinamis berasal dari keputusan/status HPA yang teramati, bukan angka tetap tiga replika. Kebijakan, metrik, rentang, serta jendela stabilisasi HPA dan penghormatan kepemilikan saat *sync* harus diverifikasi. Hasil A dilaporkan sebagai kontrol HPA, tidak digabungkan ke estimasi efikasi B–E.

F dilaporkan tersendiri karena objek di luar cakupan Application tidak otomatis menjadi sasaran prune/self-heal. Objek uji harus dibangun dari spesifikasi tersanitasi, bukan menyalin UID, resourceVersion, managedFields atau status objek server ke namespace lain. Penerimaan injeksi dan pembersihan karantina wajib dibuktikan.

G bukan kasus *self-heal* ArgoCD. Perubahan berupa anotasi metadata pada SealedSecret sintetis; pemulihan anotasi dilakukan oleh harness, bukan oleh rekonsi-

liasi ArgoCD. Nilai `spec.encryptedData` tidak dibaca atau ditulis. G dianalisis secara deskriptif dan tidak dipool ke estimasi efikasi B–E. Perubahan laten ImageUpdater dan pengendali SealedSecret harus dibekukan atau diisolasi sebelum formal.

3.5 Variabel dan Definisi Operasional

3.5.1 R1: Kesetaraan Konfigurasi

Kontrak inventaris dan bidang terpilih dibekukan dari sumber yang sama untuk kedua mode. Sumber daya cocok hanya jika ada dan seluruh bidang terpilihnya sama dengan acuan. Kontrak mencakup image, requests/limits, isi ConfigMap, kesetaraan Secret, serta bidang Service dan objek lain yang relevan.

$$R1 = \frac{\text{jumlah sumber daya dengan seluruh bidang terpilih cocok}}{\text{jumlah sumber daya dalam inventaris acuan}} \times 100\%.$$

Status Healthy/Synced bukan pengganti kesetaraan. Kesehatan dan adopsi runtime dilaporkan terpisah. Untuk Deployment yang dimiliki HPA (web dan worker), `/spec/replicas` tidak termasuk bidang terpilih R1 statis; perubahan replika skenario A karena itu tidak menurunkan R1 statis dan tidak diperlakukan sebagai drift terkelola. Status HPA (target, replika aktual, dan *event*) dilaporkan terpisah. Objek ekstra/karantina dicatat terpisah dari penyebut. Objek hilang pada pembacaan lengkap adalah ketidakcocokan; pembacaan API gagal atau bidang tidak diperiksa adalah kegagalan bukti, bukan skor nol yang sah.

R1 dicatat pada baseline, pengamatan pertama pascainjeksi, dan pascakoreksi. Jika drift sudah pulih sebelum pembacaan, kondisi sesaatnya ditandai tidak teramati. Secret dibandingkan di memori; hanya identitas bidang dan boolean kesetaraan disimpan, tanpa nilai atau hash yang dapat digunakan untuk menebak isinya.

3.5.2 R2: Deteksi, Bukti Deviasi, dan Pemulihan

Semua durasi dihitung dari satu jam monotonik pada host pencatat dalam detik dengan pecahan; UTC disimpan untuk audit. Kolektor dan injektor otomatis memakai sumber jam yang sama. Kolektor harus aktif sebelum injeksi. Sampling berlangsung pada interval nominal satu detik; interval aktual dan jeda pembacaan dicatat, terpisah dari jadwal mekanisme koreksi.

- t_0: pengiriman permintaan injeksi oleh kolektor/injektor otomatis; t_a: penerimaan respons API. Injeksi sah hanya jika API berhasil dan perubahan bidang diverifikasi dari respons atau pembacaan balik.
- t_d: sinyal deteksi pertama. Pada mode imperatif, waktu pemeriksa imperatif pertama memberi sinyal berdasarkan perbedaan konfigurasi; sumbernya dicatat. Pada mode GitOps untuk skenario B–E, waktu ArgoCD pertama menunjukkan perbedaan pada bidang terpilih. Rollout gagal bukan definisi drift; rollout sehat dapat tetap memiliki image, konfigurasi, atau limits yang salah. Sinyal HPA dicatat sebagai sinyal HPA, bukan deteksi ArgoCD.
- t_e: waktu pertama bukti tersimpan mengidentifikasi objek dan bidang yang berbeda. Pada mode imperatif, bukti berasal dari log pemeriksa imperatif, manifest diff, atau snapshot yang disimpan kolektor; pada mode GitOps, dari diff/status ArgoCD. Diff menunjukkan deviasi, bukan penyebab kausal; status Healthy dan selesainya perintah koreksi tidak menghasilkan waktu akar masalah.
- t_r: pengamatan pertama seluruh inventaris dan bidang kembali cocok, dikonfirmasi pada pembacaan berikutnya. Pada mode imperatif, konfirmasi dilakukan setelah prosedur koreksi imperatif selesai dan evaluator memverifikasi dua pembacaan; pemulihan replika oleh HPA tidak dihitung sebagai koreksi imperatif. Pada mode GitOps, konfirmasi dilakukan setelah rekonsiliasi ArgoCD. Pada F, ketiadaan objek karantina diperiksa tersendiri.

Durasi utama adalah $t_d - t_0$, $t_e - t_0$, dan $t_r - t_0$. Semua cap waktu dicatat oleh kolektor/injektor otomatis dengan jam UTC dan jam monotonik dari host pencatat yang sama; durasi dihitung dari jam monotonik, bukan selisih jam dinding. Pada mode imperatif, sinyal HPA dan respons replika diberi label pengendali HPA, bukan deteksi ArgoCD. Tidak adanya sinyal sampai batas pengamatan dicatat tersensor kanan dengan waktu hilang, tidak diisi nol atau disamakan dengan t_0. Pemulihan konfigurasi tidak membuktikan layanan sehat atau data aplikasi pulih.

3.5.3 R3: Batas Pengukuran dan Pekerjaan Lanjutan

R3 (bukti keterlacakan) tidak diukur dalam penelitian ini karena audit API server Kubernetes tidak tersedia pada lingkungan eksperimen. Tanpa audit, pelaku mutasi *out-of-band* tidak dapat direkonstruksi, sehingga klaim keterlacakan tidak dapat dibuat untuk kedua mode. Ketiadaan audit tidak menghambat *deployment* atau

pengukuran R1/R2; ia hanya menutup R3. R3 tidak dilaporkan sebagai nol, tidak dilaporkan sebagai tidak ada perbedaan, dan tidak diuji secara statistik. Unsur pelaku, objek dan bidang, waktu dan urutan, serta tindakan dan hasil didokumentasikan sebagai keterbatasan dan pekerjaan lanjutan setelah audit diaktifkan.

3.6 Gerbang Baseline dan Reset

Sebelum injeksi, inventaris sumber daya, pod dan node wajib tidak kosong serta lengkap; semua pod layanan harus Running dan Ready. CPU dan memori setiap node harus di bawah 70%. Memori didefinisikan sebagai working set metrics-server dibagi allocatable node, bukan metrik host yang dapat dipertukarkan. Metrik tidak tersedia, field tidak terbaca, inventaris parsial, atau status pengendali tidak diketahui menggagalkan gerbang secara tertutup. Kelulusan sampel tunggal tidak menggantikan bukti jendela beban stabil.

Hanya satu stack InvenioRDM yang boleh aktif. Sebelum mode dimulai, tidak boleh ada namespace aplikasi kembar atau objek sisa dari blok sebelumnya. Untuk mode imperatif, *auto-sync/self-heal* pada `apps` dan `invenio-bootstrap` harus terverifikasi jeda sebelum injeksi; untuk mode GitOps, keduanya harus terverifikasi aktif. Reset wajib memverifikasi: (1) salinan seed privat `4ben9-bgh38` di namespace `invenio` dengan `seed.bin` 65.536 byte dan SHA-256 yang cocok masih ada; (2) tidak ada record, draft, atau berkas lain di luar seed; (3) konfigurasi, Secret, dan antrean kembali ke acuan; (4) HPA berada pada keadaan normal; (5) semua pod Running dan Ready; serta (6) tidak ada objek karantina. Reset dinyatakan sah hanya setelah dua pembacaan lengkap yang berurutan cocok. Rollout sehat saja tidak cukup. Kegagalan satu tindakan koreksi diteruskan ke pemanggil dan pengulangan berikutnya dilarang sampai reset lengkap terverifikasi. Tidak ada pengosongan queue, modifikasi data, restore, atau migrasi langsung yang diizinkan oleh naskah ini.

Kesiapan juga mencakup sinkronisasi jam, kepemilikan HPA, beban latar, serta pemulihan aplikasi/data/queue yang telah ditinjau. Audit API belum tersedia; ini tidak menjadi gerbang *deployment*, tetapi menutup R3. Pemulihan aplikasi dan data setelah seed belum diuji; *private-draft seed* hanya membuktikan jalur unggah/unduh, bukan pemulihan record dan berkas yang konsisten. Uji pemulihan pasca-seed tetap menjadi prasyarat. Sampel kapasitas 25 Oktober 2026 menunjukkan 59% memori pada kedua worker, tetapi belum ada jendela berkelanjutan atau bukti penempatan dua stack; karena itu mode dijalankan berurutan pada satu stack.

SMTP masih memakai placeholder `smtp.example.org` dan `admin@example.org`; relay, autentikasi, CA, dan firewall belum diketahui, sehingga verifikasi email serta notifikasi email ditunda dan tidak menjadi bagian dari R1–R2. OpenSearch berjalan sebagai satu node; HA OpenSearch ditunda dan tidak diklaim. Tidak ada skenario yang menguji email atau HA, sehingga kedua hal tersebut berada di luar cakupan revisi ini.

Bootstrap, migrasi, fixtures, indexing dan deklarasi queue harus dipisahkan dari repair rutin. Sync perbaikan yang memicu setup dapat menambah kerja dan mengubah baseline. Antarmuka lifecycle wajib mengembalikan exit code, waktu, identitas tindakan dan postcondition; kegagalan tak terduga tidak boleh ditelan. ConfigMap/Secret yang dipakai melalui envFrom memerlukan bukti adopsi runtime terpisah, bukan hanya kesetaraan objek.

3.7 Instrumen dan Protokol Pengukuran

Empat entrypoint `baseline-check.sh`, `drift-inject.sh`, `metrics-collect.sh`, dan `recovery-runbook.sh` kini memvalidasi bundel bukti tersimpan secara offline. Pemeriksa bersama menolak bukti hilang/tidak konsisten, menggunakan aturan R1 identik, membedakan sinyal pengendali dari rollout, serta mempertahankan waktu tersensor. Uji fixture mencakup sukses, kegagalan dan batas; fixture bukan hasil pengukuran aplikasi. Adapter polling, injeksi, pemulihan/reset langsung, ekspor render, integrasi lifecycle, dan SMTP belum selesai; pengujian formal tidak boleh dimulai sebelum adapter tersebut ditinjau dan divalidasi. Protokol otomasi imperatif, kolektor pasif, aturan atribusi pengendali, dan rencana 210 run dirinci pada `experiments/LIVE-OBSERVER.md`; dokumen tersebut adalah rencana, bukan prosedur yang telah diuji.

Urutan formal yang direncanakan ialah: (1) bekukan metode, sumber, batas waktu dan aturan analisis; (2) pilih mode dan urutan yang tercatat; (3) bekukan revisi sumber, render SHA-256, image digest, dan versi runtime; (4) verifikasi baseline dan reset ke seed bersih; (5) aktifkan kolektor dan pemeriksa imperatif atau ArgoCD sesuai mode; (6) kirim satu injeksi dan verifikasi penerimaan; (7) catat sinyal pengendali, bukti deviasi, kesetaraan dan kesehatan sampai pulih atau batas pengamatan; (8) simpan kegagalan dan hasil tersensor; (9) verifikasi reset lengkap dan pemulihan ArgoCD sebelum pengulangan berikutnya atau perpindahan mode. Tidak ada eksekusi formal dalam revisi ini.

3.8 Pengulangan dan Analisis Statistik

Sampel formal adalah 15 pengulangan per skenario per mode: 10 pengulangan protokol R2 dan 5 pengulangan protokol R1, sehingga totalnya $7 \times 2 \times 15 = 210$ run pada satu kasus. Pembacaan R1 tetap disimpan pada setiap run sebagai bukti status, sedangkan 5 pengulangan R1 diarahkan pada protokol konsistensi. F dan G tetap menjalani protokol yang sama, tetapi hasil waktunya dilaporkan deskriptif dan tidak dipool ke estimasi efikasi. Sampling berlangsung pada interval nominal satu detik. Sebelum run, revisi sumber, SHA-256 hasil render, image digest, dan versi runtime dibekukan. Target 15 bukan hasil analisis daya dan tidak menjamin kecukupan daya per skenario (Dybå et al., 2006; Kitchenham and Madeyski, 2024; Cohen, 1988).

Mode dijalankan sebagai blok berurutan; urutan mode diacak dan dicatat, dan urutan skenario memakai permutasi yang sama di kedua mode. Karena mode tidak serentak dan memakai stack yang sama, kondisi klaster antarblok dapat berbeda dan observasi tidak diperlakukan independen secara naif. A adalah kontrol HPA; F batas visibilitas; G kasus deskriptif bukan *self-heal* ArgoCD. Ketiganya dilaporkan terpisah dari B–E dan tidak dipool ke estimasi efikasi. Menjumlahkan skenario yang heterogen tidak membuktikan daya uji per skenario.

Laporkan jumlah injeksi sah, kejadian teramati/tersensor, kegagalan instrumen/pemulihan/reset, median, rentang dan distribusi per skenario. Waktu tidak teramati tetap kosong dengan status sensor dan batas pengamatan. Kegagalan API/injeksi/infrastruktur dibedakan dari tidak terjadinya deteksi; data tidak dibuang tanpa jejak. Analisis eksploratif memakai uji nonparametrik berpasangan/blok (misalnya Wilcoxon *signed-rank*) bila pemasangan skenario-pengulangan dapat dipertahankan, atau *Mann–Whitney U* dua sisi bila tidak, dengan $\alpha = 0{,}05$ dan koreksi Holm untuk keluarga uji yang ditetapkan sebelum data. Ukuran efek (Cliff's δ atau Hodges–Lehmann) dilaporkan bersama interval kepercayaan bootstrap. Waktu deteksi dan pemulihan yang tersensor kanan dianalisis dengan Kaplan–Meier serta median/*restricted mean survival*; nilai tersensor tidak diimputasi nol. Analisis yang mengabaikan penyensoran tidak dianggap perbandingan lengkap. Rincian keluarga uji dan penanganan penyensoran memerlukan tinjauan pemilik/pembimbing sebelum formal.

3.9 Ancaman terhadap Validitas

3.9.1 Validitas Konstruk

R1 mengukur kesetaraan bidang terpilih terhadap acuan, bukan kebenaran semantik, keamanan, atau kelengkapan seluruh konfigurasi. R2 mengukur waktu pengamatan pada resolusi satu detik, bukan waktu kejadian kontinu; sinyal deteksi, bukti deviasi, dan pemulihan konfigurasi dibedakan. Kesehatan layanan dan adopsi runtime dilaporkan terpisah. R3 tidak diukur karena audit API tidak tersedia, sehingga klaim keterlacakan tidak dapat dibuat. A, F, dan G tidak boleh digunakan untuk klaim efikasi *self-heal* ArgoCD.

3.9.2 Validitas Internal

Validitas internal dipengaruhi perbedaan jadwal pengendali, tekanan node, I/O latar, dependensi bersama, hooks lifecycle, dan kepemilikan HPA/SealedSecret/ImageUpdater. Karena kedua mode berurutan pada stack yang sama, tidak ada isolasi penuh: beban latar dan keadaan klaster dapat berubah antarb lok. Urutan mode diacak, kondisi dicatat, reset diverifikasi, dan penjedaan/pemulihan ArgoCD diperiksa, tetapi perbedaan waktu tidak dapat dieliminasi sepenuhnya. Tidak ada waktu reaksi manusia yang diukur; kedua mode berjalan otomatis. Resolusi kolektor serta interval pemeriksa imperatif dan rekonsiliasi ArgoCD membatasi ketepatan waktu. Kendali dan pencatatan tersebut membatasi bias, tidak menjamin satu-satunya perbedaan adalah paradigma abstrak.

3.9.3 Validitas Eksternal

Validitas eksternal dibatasi satu klaster, satu stack InvenioRDM produksi-like, satu implementasi ArgoCD, satu implementasi otomasi imperatif, dan konfigurasi yang akhirnya digunakan. Studi ini adalah studi kasus tunggal; hasilnya tidak digeneralisasi ke seluruh aplikasi multi-service, klaster, atau alat GitOps. Surrogat S1/S2 serta hasil mock historis bukan sampel formal dan tidak memperluas inferensi.

3.9.4 Validitas Kesimpulan

Jumlah pengulangan per sel terbatas (15 run; 10 untuk R2 dan 5 untuk R1) dan tidak dijamin memiliki daya statistik per skenario. Uji nonparametrik, ukuran efek,

interval kepercayaan, dan pelaporan penyensoran dipakai agar kesimpulan tidak bergantung pada asumsi normalitas atau independensi yang tidak terpenuhi. Kegagalan, data tersensor, dan ketidakpastian dilaporkan; tidak ada klaim ekivalensi atau superioritas universal. Sampai tinjauan metode serta seluruh gerbang dipenuhi, belum tersedia dasar untuk kesimpulan komparatif empiris.

BAB IV
JADWAL PENELITIAN

Penelitian direncanakan untuk dilaksanakan selama enam bulan, dari bulan Juni 2026 hingga November 2026. Jadwal penelitian disusun berdasarkan tahapan studi kasus komparatif terkontrol: studi literatur, perancangan apparatus satu stack, pengembangan skrip otomasi, kalibrasi *baseline*, pelaksanaan eksperimen, analisis statistik, dan penulisan laporan. Tahapan-tahapan tersebut dipersempit dan saling tumpang tindih untuk efisiensi waktu.

Tabel 4.1 menunjukkan rencana jadwal pelaksanaan penelitian berbasis bulan. Simbol • menunjukkan bahwa kegiatan dilaksanakan pada bulan tersebut.

Tabel 4.1: Jadwal Penelitian (Gantt Chart)

Kegiatan	Bulan					
	Jun	Jul	Agu	Sep	Okt	Nov
Studi Literatur & Perancangan Apparatus	•	•				
Implementasi ArgoCD, Mode Imperatif, dan *Baseline*		•	•			
Pengembangan Skrip Otomasi		•	•			
Kalibrasi *Baseline* & Uji Coba			•	•		
Eksperimen & Pengumpulan Data (E1–E2, 210 run)					•	•
Analisis Statistik & Interpretasi						•
Penulisan Laporan	•	•	•	•	•	•
Seminar / Sidang						•

4.1 Penjelasan Kegiatan

1. Studi Literatur dan Perancangan Apparatus (Juni – Juli 2026)

   Kegiatan ini mencakup pengumpulan dan analisis literatur mengenai penyimpangan konfigurasi pada Kubernetes, evaluasi empiris GitOps, metode studi kasus dan eksperimen pada riset sistem, serta klasifikasi *workload* multi-service. Sumber literatur meliputi jurnal ilmiah (IEEE, ACM, Springer), buku teks, dokumentasi resmi, dan artikel industri yang ditandai sebagai non-*peer-review*. Tahap ini juga mencakup perancangan dua mode pengelolaan pada satu stack InvenioRDM produksi-like, definisi variabel dan intervensi, pemilihan tujuh skenario penyimpangan, serta spesifikasi protokol pengukuran. Tidak ada stack

kembar `exp-*` dan tidak ada eksekusi paralel A/B dalam rancangan ini.

2. Implementasi ArgoCD, Mode Imperatif, dan *Baseline* (Juli – Agustus 2026)

   Tahap implementasi meliputi verifikasi ArgoCD pada stack InvenioRDM yang ada, prosedur penjedaan dan pemulihan *auto-sync*/*self-heal*, pemeriksa dan *runbook* imperatif terotomasi (*workflow* `kubectl` otomatis), serta konfigurasi *logging* dan *monitoring* (Prometheus, Grafana, ArgoCD *notifications*). Pada akhir fase ini, kedua mode siap dijalankan berurutan pada stack yang sama; hanya satu mode aktif pada satu waktu. Audit API server tidak diaktifkan pada fase ini dan tidak menjadi gerbang *deployment*.

3. Pengembangan Skrip Otomasi (Juli – Agustus 2026)

   Pengembangan empat skrip otomasi yang mendukung konsistensi prosedural eksperimen: (a) `drift-inject.sh` untuk injeksi skenario penyimpangan secara acak dengan pencatatan *timestamp*; (b) `metrics-collect.sh` untuk pengumpulan metrik secara berkala via ArgoCD API dan `kubectl`; (c) `baseline-check.sh` untuk verifikasi kondisi klaster sebelum setiap *run*; dan (d) `recovery-runbook.sh` untuk memverifikasi hasil koreksi pada mode imperatif. Skrip dikembangkan bersama implementasi apparatus.

4. Kalibrasi *Baseline* dan Uji Coba (Agustus – September 2026)

   Sebelum eksperimen dimulai, fase kalibrasi dilakukan untuk memastikan validitas internal. Kegiatan meliputi: (a) *deployment* beban uji (*workload*) komposit multi-service pada satu stack dan verifikasi kedua mode secara berurutan; (b) pembekuan revisi sumber, SHA-256 hasil render, image digest, dan versi runtime; (c) verifikasi *baseline* bersih dan reset; (d) verifikasi kepemilikan HPA serta prosedur penjedaan/pemulihan ArgoCD; (e) verifikasi penggunaan sumber daya *node* di bawah 70%; dan (f) uji coba protokol kolektor dengan sampling satu detik serta skrip otomasi. Pilot terbatas bukan data formal.

5. Eksperimen dan Pengumpulan Data (Oktober – November 2026)

   Rencana pelaksanaan dua kelompok eksperimen menunggu tinjauan metode dan kelulusan gerbang Bab III. E1 menilai kesetaraan konfigurasi (R1) dan E2 menilai waktu deteksi, bukti deviasi, dan pemulihan (R2). Sampel formal adalah 15 pengulangan per skenario per mode, yaitu 10 pengulangan protokol R2

dan 5 pengulangan protokol R1, sehingga totalnya $7 \times 2 \times 15 = 210$ run pada satu kasus. Sampling berlangsung pada interval nominal satu detik. Target tersebut bukan jaminan daya uji. Kedua mode dijalankan berurutan pada satu stack; urutan mode diacak dan dicatat, sedangkan urutan skenario memakai permutasi yang sama. A dilaporkan sebagai kontrol HPA, F sebagai batas visibilitas, dan G sebagai kasus deskriptif bukan *self-heal* ArgoCD; ketiganya terpisah dari efikasi B–E. Satu injeksi diikuti reset lengkap; kegagalan dan hasil tersensor tetap disimpan. R3 tidak diukur karena audit API server tidak tersedia. Eksekusi formal belum diizinkan.

6. Analisis Statistik dan Interpretasi (November 2026)

   Analisis data meliputi statistik deskriptif (*mean*, median, rentang, deviasi standar), uji nonparametrik berpasangan/blok bila pemasangan dapat dipertahankan atau *Mann–Whitney U* bila tidak, perhitungan ukuran efek (Cliff's δ atau Hodges–Lehmann) beserta interval kepercayaan, serta analisis *survival* untuk waktu deteksi dan pemulihan yang tersensor kanan. Pembahasan mencakup skenario penyimpangan yang paling berdampak, faktor yang memengaruhi hasil, non-isolasi pada satu stack, serta perumusan informasi kontekstual, bukan rekomendasi universal pemilihan paradigma.

7. Penulisan Laporan (Juni – November 2026)

   Penulisan laporan dilakukan secara paralel sepanjang penelitian. Setiap tahap menghasilkan bab atau bagian yang sesuai. Draft proposal diselesaikan pada pertengahan periode, draft skripsi lengkap pada akhir penelitian.

8. Seminar / Sidang (November 2026)

   Presentasi dan sidang tugas akhir di depan dosen penguji, mencakup paparan hasil eksperimen, analisis statistik, informasi kontekstual, dan kesimpulan yang dibatasi pada kasus yang diuji.

DAFTAR PUSTAKA

R. Aghili, Q. Qin, H. Li, and F. Khomh. Understanding web application workloads and their applications: Systematic literature review and characterization. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024.

Argo Project. Argocd documentation, 2024. URL `https://argo-cd.readthedocs.io`.

Argo Project. Argo cd documentation: Automated sync policy. `https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/`, 2025. Dokumentasi resmi perangkat lunak.

Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1):70–93, 2016.

Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.

Cloud Native Computing Foundation. Cncf cloud native definition v1.0, 2024. URL `https://www.cncf.io/about/who-we-are`.

CNCF GitOps Working Group. Opengitops principles v1.0.0. `https://opengitops.dev`, 2021.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.

Tore Dybå, Vigdis By Kampenes, and Dag I. K. Sjøberg. A systematic review of statistical power in software engineering experiments. *Information and Software Technology*, 48(8):745–755, 2006. doi: 10.1016/j.infsof.2005.08.009.

Christof Ebert, Gorka Gallardo, Josune Hernantes, and Nicolas Serrano. Devops. *IEEE Software*, 33(3):94–100, 2016. doi: 10.1109/MS.2016.68.

Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

Md Mahadi Hassan, John Salvador, Shubhra Kanti Karmaker Santu, and Akond Rahman. State reconciliation defects in infrastructure as code. *Proceedings of the ACM on Software Engineering*, 1(FSE):1865–1888, 2024. doi: 10.1145/3660790.

Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.

Barbara Kitchenham and Lech Madeyski. Recommendations for analysing and meta-analysing small sample size software engineering experiments. *Empirical Software Engineering*, 29(6), 2024. doi: 10.1007/s10664-024-10504-1.

Thomas A. Limoncelli, Strata R. Chalup, and Christina J. Hogan. *The Practice of Cloud System Administration: Designing and Operating Large Distributed Systems*. Addison-Wesley Professional, 2nd edition, 2018.

Gerardo Lopez and Saloni Narang. Gitops in 2025: From old-school updates to the modern way. `https://www.cncf.io/blog/2025/06/09/gitops-in-2025-from-old-school-updates-to-the-modern-way/`, 2025. Artikel industri CNCF, bukan artikel peer-review; hanya untuk konteks.

M. E. Matubber. Managing 5g kubernetes infrastructures using gitops principles. Master's thesis, KTH Royal Institute of Technology, 2025.

Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

P. Kagganti Nataraja. A security-centric analysis of declarative and imperative deployment approaches in kubernetes-based application environments. Master's thesis, National College of Ireland, 2025.

O. Pohjola. Mitigating configuration drift in infrastructure-as-code systems. Master's thesis, Aalto University, 2025.

A. Rahman, S. I. Shamim, D. B. Bose, et al. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 2023.

Akond Rahman and Laurie Williams. Source code properties of defective infrastructure as code scripts. *Information and Software Technology*, 112:148–163, 2019. doi: 10.1016/j.infsof.2019.04.013.

Akond Rahman, Chris Parnin, and Laurie Williams. The seven sins: Security smells in infrastructure as code scripts. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 164–175. IEEE/ACM, 2019. doi: 10.1109/ICSE.2019.00033.

Akond Rahman, Effat Farhana, Chris Parnin, and Laurie Williams. Gang of eight: A defect taxonomy for infrastructure as code scripts. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering (ICSE)*. ACM/IEEE, 2020. doi: 10.1145/3377811.3380409.

Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.

Marc Sallin, Martin Kropp, Craig Anslow, James W. Quilty, and Andreas Meier. Measuring software delivery performance using the four key metrics of devops. In *Agile Processes in Software Engineering and Extreme Programming (XP 2021)*, pages 103–119. Springer, 2021. doi: 10.1007/978-3-030-78098-2_7.

D. Schlette, P. Empl, M. Caselli, T. Schreck, and G. Pernul. Do you play it by the books? a study on incident response playbooks and influencing factors. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024.

Michael L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4th edition, 2015.

R. Shrestha and A. A. N. Ali. Configuration management in kubernetes environments: A gitops approach. In *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*. IEEE, 2024.

H. Siddiqui, F. Khendek, and M. Toeroe. Microservices based architectures for iot systems—state-of-the-art review. *Internet of Things*, 2023.

Ervina Utami and Hanif Al Fatta. Analysis on the use of declarative and pull-based deployment models on gitops using argo cd. In *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. IEEE, 2021.

Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15)*, pages 1–17. ACM, 2015.

Brennan Wilkes, Alessandra Maciel Paz Milani, and Margaret-Anne Storey. A framework for automating the measurement of devops research and assessment (dora) metrics. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2023. doi: 10.1109/ICSME58846.2023.00018.

Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2nd edition, 2012.

Billy Yuen, Alexander Matyushentsev, Jesse Suen, and Thomas Ekenstam. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*. O'Reilly Media, 2021. ISBN 978-1492094877.

Yue Zhang, Uchswas Paul, Marcelo d'Amorim, and Akond Rahman. Configuration defects in kubernetes. *IEEE Transactions on Software Engineering*, 52(3):1125–1144, 2026. doi: 10.1109/TSE.2026.3659785.